

Typing Rules, Validity, Injectivity, and Subject Reduction for Dependent Type Theory with Π , Σ , $\mathbf{U}:\mathbf{U}$, and $\mathbf{Prop}:\mathbf{U}$

1 Judgments

We work with three mutually defined judgment forms:

$\Gamma \text{ ctx}$	Γ is a well-formed context
$\Gamma \vdash M : A$	M has type A in context Γ
$\Gamma \vdash M = N : A$	M and N are definitionally equal at type A

Contexts are snoc-lists: $\Gamma ::= \cdot \mid \Gamma, x:A$. We write $M\sigma$ for the result of applying a substitution σ to a term M , and $[t]$ for the substitution $\Gamma \rightarrow \Gamma, x:A$ extending the identity by t , so that $B[a]$ means $B(\text{id}, a)$.

2 Well-formed contexts

$$\frac{}{\cdot \text{ ctx}} \qquad \frac{\Gamma \vdash A : \mathbf{U}}{\Gamma, x:A \text{ ctx}}$$

3 Typing rules

Core rules

$$\frac{\text{VAR} \quad \Gamma \text{ ctx} \quad (x:A) \in \Gamma}{\Gamma \vdash x : A} \qquad \frac{\text{CONV} \quad \Gamma \vdash M : A \quad \Gamma \vdash A = B : \mathbf{U} \quad \Gamma \vdash B : \mathbf{U}}{\Gamma \vdash M : B} \qquad \frac{\text{U} \quad \Gamma \text{ ctx}}{\Gamma \vdash \mathbf{U} : \mathbf{U}}$$

Pi types

$$\frac{\text{PI} \quad \Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U}}{\Gamma \vdash \Pi(x:A)B : \mathbf{U}} \qquad \frac{\text{LAM} \quad \Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U} \quad \Gamma, x:A \vdash M : B}{\Gamma \vdash \lambda(x:A)M : \Pi(x:A)B}$$

$$\frac{\text{APP} \quad \Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U} \quad \Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B[a]}$$

Sigma types

$$\frac{\text{SIGMA} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U}}{\Gamma \vdash \Sigma(x:A)B : \mathbb{U}} \quad \frac{\text{MKPAIR} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : A \quad \Gamma \vdash N : B[M]}{\Gamma \vdash \text{pair}(M, N) : \Sigma(x:A)B}$$

$$\frac{\text{FST} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : \Sigma(x:A)B}{\Gamma \vdash \text{fst}(M) : A}$$

$$\frac{\text{SND} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : \Sigma(x:A)B}{\Gamma \vdash \text{snd}(M) : B[\text{fst}(M)]}$$

Propositions

$$\frac{\text{PROP} \quad \Gamma \text{ ctx}}{\Gamma \vdash \text{Prop} : \mathbb{U}} \quad \frac{\text{PROP-U} \quad \Gamma \vdash A : \text{Prop}}{\Gamma \vdash A : \mathbb{U}} \quad \frac{\text{PI-PROP} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \text{Prop}}{\Gamma \vdash \Pi(x:A)B : \text{Prop}}$$

4 Conversion rules

General

$$\frac{\text{REFL} \quad \Gamma \vdash M : A}{\Gamma \vdash M = M : A} \quad \frac{\text{SYM} \quad \Gamma \vdash M = N : A}{\Gamma \vdash N = M : A} \quad \frac{\text{TRANS} \quad \Gamma \vdash M = N : A \quad \Gamma \vdash N = P : A}{\Gamma \vdash M = P : A}$$

$$\frac{\text{CONV-EQ} \quad \Gamma \vdash M = N : A \quad \Gamma \vdash A = B : \mathbb{U} \quad \Gamma \vdash B : \mathbb{U}}{\Gamma \vdash M = N : B}$$

Pi computation and congruence

$$\frac{\text{BETA} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma, x:A \vdash M : B \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda(x:A)M) a = M[a] : B[a]}$$

$$\frac{\text{PI-CONG} \quad \Gamma \vdash A = A' : \mathbb{U} \quad \Gamma, x:A \vdash B = B' : \mathbb{U}}{\Gamma \vdash \Pi(x:A)B = \Pi(x:A')B' : \mathbb{U}}$$

$$\frac{\text{FUNEXT} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma, x:A \vdash f^+ x = g^+ x : B \quad \Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash g : \Pi(x:A)B}{\Gamma \vdash f = g : \Pi(x:A)B}$$

$$\frac{\text{APP-FUN} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash f = f' : \Pi(x:A)B \quad \Gamma \vdash a : A}{\Gamma \vdash f a = f' a : B[a]}$$

$$\frac{\text{APP-ARG} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash a = a' : A}{\Gamma \vdash f a = f a' : B[a]}$$

Sigma computation and congruence

$$\frac{\text{SIGMA-CONG} \quad \Gamma \vdash A = A' : \mathbb{U} \quad \Gamma, x:A \vdash B = B' : \mathbb{U}}{\Gamma \vdash \Sigma(x:A)B = \Sigma(x:A')B' : \mathbb{U}}$$

$$\frac{\text{BETA-FST} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : A \quad \Gamma \vdash N : B[M]}{\Gamma \vdash \text{fst}(\text{pair}(M, N)) = M : A}$$

$$\frac{\text{BETA-SND} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : A \quad \Gamma \vdash N : B[M]}{\Gamma \vdash \text{snd}(\text{pair}(M, N)) = N : B[M]}$$

$$\frac{\text{PAIR-ETA} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : \Sigma(x:A)B}{\Gamma \vdash \text{pair}(\text{fst}(M), \text{snd}(M)) = M : \Sigma(x:A)B}$$

$$\frac{\text{MKPAIR-FST} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M = M' : A \quad \Gamma \vdash N : B[M] \quad \Gamma \vdash N' : B[M']}{\Gamma \vdash \text{pair}(M, N) = \text{pair}(M', N') : \Sigma(x:A)B}$$

$$\frac{\text{MKPAIR-SND} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : A \quad \Gamma \vdash N = N' : B[M]}{\Gamma \vdash \text{pair}(M, N) = \text{pair}(M, N') : \Sigma(x:A)B}$$

$$\frac{\text{FST-CONG} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M = M' : \Sigma(x:A)B}{\Gamma \vdash \text{fst}(M) = \text{fst}(M') : A}$$

$$\frac{\text{SND-CONG} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M = M' : \Sigma(x:A)B}{\Gamma \vdash \text{snd}(M) = \text{snd}(M') : B[\text{fst}(M)]}$$

Propositions

$$\frac{\text{PROP-IRR} \quad \Gamma \vdash A : \text{Prop} \quad \Gamma \vdash M : A \quad \Gamma \vdash N : A}{\Gamma \vdash M = N : A}$$

$$\frac{\text{PROP-U-EQ} \quad \Gamma \vdash M = N : \text{Prop}}{\Gamma \vdash M = N : \mathbb{U}}$$

$$\frac{\text{PI-PROP-CONG} \quad \Gamma \vdash A = A' : \mathbb{U} \quad \Gamma, x:A \vdash B = B' : \text{Prop}}{\Gamma \vdash \Pi(x:A)B = \Pi(x:A')B' : \text{Prop}}$$

5 Head reduction

We define single-step head reduction $M \rightsquigarrow_1 N$ as the compatible closure of β -reduction at the head, extended with projection reductions for Sigma types:

$$\begin{array}{c}
\text{HR-BETA} \\
\hline
(\lambda(x:A)M) a \rightsquigarrow_1 M[a]
\end{array}
\quad
\begin{array}{c}
\text{HR-APP} \\
\hline
M \rightsquigarrow_1 M' \\
\hline
M a \rightsquigarrow_1 M' a
\end{array}
\quad
\begin{array}{c}
\text{HR-BETA-FST} \\
\hline
\text{fst}(\text{pair}(M, N)) \rightsquigarrow_1 M
\end{array}
\quad
\begin{array}{c}
\text{HR-BETA-SND} \\
\hline
\text{snd}(\text{pair}(M, N)) \rightsquigarrow_1 N
\end{array}$$

$$\begin{array}{c}
\text{HR-FST} \\
\hline
M \rightsquigarrow_1 M' \\
\hline
\text{fst}(M) \rightsquigarrow_1 \text{fst}(M')
\end{array}
\quad
\begin{array}{c}
\text{HR-SND} \\
\hline
M \rightsquigarrow_1 M' \\
\hline
\text{snd}(M) \rightsquigarrow_1 \text{snd}(M')
\end{array}$$

We write $M \rightsquigarrow^* N$ for the reflexive-transitive closure of $\rightsquigarrow_1$. As in the Pi-only system, the validity relation uses a wrapper $\text{Red } \Gamma M N A$ with phantom context and type parameters.

6 Domain model

We describe the domain-theoretic semantics following Coquand–Huber (2018). The model interprets types as *codes* (finite elements of a Scott domain) and terms as elements approximated by these codes.

6.1 Finite elements

The set Code of *codes* is defined inductively, together with a set FunCode of *finite functions* (finite lists of input-output pairs):

$$\begin{aligned}
u, v, a &::= \perp \mid \mathbf{U} \mid \mathbf{Prop} \mid \lambda g \mid \text{Pi}(a, f) \mid \text{Sigma}(a, f) \mid (u, v) \\
g, f, h &::= [] \mid (u, v) :: g
\end{aligned}$$

Here $\perp$ is the bottom element (no information), $\mathbf{U}$ and $\mathbf{Prop}$ are the codes for the universe and the propositional universe, λg is a function value with finite graph g , $\text{Pi}(a, f)$ is a Pi-type code with domain code a and codomain function f , $\text{Sigma}(a, f)$ is a Sigma-type code, and (u, v) is a pair code.

6.2 Evaluation

The evaluation of a finite function f at input u computes the *supremum* of all outputs whose keys are below u in the information ordering:

$$\text{Eval}(f, u) = \bigsqcup \{v_i \mid (u_i, v_i) \in f, u_i \leq u\}$$

The binary supremum $u \sqcup v$ combines compatible elements: $\perp \sqcup u = u$, $\mathbf{U} \sqcup \mathbf{U} = \mathbf{U}$, $(\lambda g) \sqcup (\lambda h) = \lambda(g ++ h)$, $\text{Pi}(a, f) \sqcup \text{Pi}(b, g) = \text{Pi}(a \sqcup b, f ++ g)$, and $u \sqcup v = \perp$ when u and v have different top-level constructors.

6.3 Information ordering

The ordering $u \leq v$ (written $\text{LeCode } u v$ in the formalisation) is defined by structural recursion. On base codes, $\perp$ is below everything, $\mathbf{U} \leq \mathbf{U}$, and $\mathbf{Prop} \leq \mathbf{Prop}$. On compound codes:

$$\begin{aligned}
\lambda g &\leq \lambda h \quad \text{iff} \quad \text{for each } (u_i, v_i) \in g, \quad v_i \leq \text{Eval}(h, u_i) \\
\text{Pi}(a, f) &\leq \text{Pi}(b, g) \quad \text{iff} \quad a \leq b \quad \text{and} \quad \text{for each } (u_i, v_i) \in f, \quad v_i \leq \text{Eval}(g, u_i)
\end{aligned}$$

Codes with different top-level constructors are incomparable. The ordering on finite functions uses the *finite condition*: $g \leq h$ iff for each $(u, v) \in g$, $v \leq \text{Eval}(h, u)$. For coherent functions, this is equivalent to pointwise ordering $\forall u. \text{Eval}(g, u) \leq \text{Eval}(h, u)$.

6.4 Coherence

A finite function $g = [(u_1, v_1), \dots, (u_n, v_n)]$ represents a well-defined continuous function only if it is *coherent*: whenever two keys are *compatible* (have a common upper bound), their values must also be compatible. Formally, $\text{Comp}(u, v)$ holds when u and v can be combined via the supremum (same constructor, recursively compatible), and $\text{CoherentFun}(g)$ requires:

- each key u_i and value v_i is coherent (recursively),
- each value v_i is non- $\perp$,
- for all $i < j$, if $\text{Comp}(u_i, u_j)$ then $\text{Comp}(v_i, v_j)$.

On compound elements: $\text{Coherent}(\text{Pi}(a, f))$ requires $\text{Coherent}(a)$ and $\text{CoherentFun}(f)$, and similarly for $\text{Sigma}(a, f)$. Base codes ($\perp$, U , Prop) are always coherent.

Coherence ensures that the supremum-based evaluation is monotone: if $u \leq v$ and g is coherent, then $\text{Eval}(g, u) \leq \text{Eval}(g, v)$.

6.5 Finite membership

The predicate $\text{FinMem}(u, a)$ asserts that code u is a valid *approximation to an element of type a* . Its key clauses are:

$$\begin{aligned} \text{FinMem}(\perp, a) &= \text{FinMem}(a, \text{U}) \\ \text{FinMem}(\text{U}, \text{U}) &= \top \quad \quad \quad \text{FinMem}(\text{Prop}, \text{U}) = \top \\ \text{FinMem}(\text{Pi}(a, f), \text{U}) &= \text{FinMem}(a, \text{U}) \wedge \text{FinMemAllU}(f, a) \wedge \text{CoherentFun}(f) \\ \text{FinMem}(\lambda g, \text{Pi}(a, f)) &= \text{FinMemFun}(g, a, f) \wedge \text{CoherentFun}(g) \wedge \text{FinMem}(\text{Pi}(a, f), \text{U}) \end{aligned}$$

Here $\text{FinMemFun}(g, a, f)$ requires that for each $(u_i, v_i) \in g$, $\text{FinMem}(u_i, a)$ and $\text{FinMem}(v_i, \text{Eval}(f, u_i))$; and $\text{FinMemAllU}(f, a)$ requires that each pair in f has its key in a and its value in U . The clause $\text{FinMem}(\perp, a) = \text{FinMem}(a, \text{U})$ means that $\perp$ is a valid approximation in any well-formed type.

6.6 Key properties

The following lemmas, all proved in the Agda formalisation without postulates, are central to the adequacy proof:

- **Sup closure:** $\text{FinMem}(u, a)$ and $\text{FinMem}(v, a)$ imply $\text{FinMem}(u \sqcup v, a)$.
- **Coherent evaluation:** If $\text{Coherent}(\lambda f)$ and $\text{Coherent}(u)$, then $\text{Coherent}(\text{Eval}(f, u))$.
- **Monotonicity of FinMem:** If $u \leq v$ and $\text{FinMem}(v, a)$, then $\text{FinMem}(u, a)$.
- **Comp-down:** If $u \leq u'$ and $\text{Comp}(u', v)$, then $\text{Comp}(u, v)$.

6.7 Termination

All definitions in the mutual block (`LeCode`, `LeFunCode`, `Eval`, `Comp`, `CompFun`, `Coherent`, `FinMem`, and their auxiliaries) are terminating, although Agda’s termination checker requires a `TERMINATING` pragma for the non-structural cases. We sketch the informal arguments.

LeCode and Comp. These are defined by pattern-matching on the pair of constructors. Same-constructor cases recurse structurally: $\text{Pi}(a, f) \leq \text{Pi}(b, g)$ recurses on (a, b) and on `LeFunCode`(f, g), which in turn checks $v_i \leq \text{Eval}(g, u_i)$ for each $(u_i, v_i) \in f$. The result `Eval`(g, u_i) is built from values of entries of g glued by $\sqcup$; it is bounded in size by g . Meanwhile v_i is a strict subterm of the first argument. So all recursive calls operate on strictly smaller data in the total-size measure `rk`. Cross-constructor cases return $\perp$ (empty type) immediately. The same argument applies verbatim to `Comp` and `CompFun`.

Coherent. `Coherent`(u) is defined by structural recursion on u . On `Pi`(a, f) it recurses into `Coherent`(a) and `CoherentFunTail`(f), which walks the list f checking `Coherent` on each key and value (strict subterms) and `CoherentWith` between each entry and the tail (which calls `Comp`, already shown terminating). No non-structural step occurs.

FinMem. The only non-structural clause is `FinMem`($\perp, a$) = `FinMem`(a, U), which swaps the arguments. This can fire *at most once*: after the swap, the first argument is a (the original second argument), and every clause for a non- $\perp$ first argument either returns $\top/\perp$ immediately, or recurses on strict subterms of both arguments. In particular:

- If $a = \perp$, we get `FinMem`($\perp, \text{U}$) = `FinMem`(U, U) = $\top$.
- If a is non- $\perp$, the subsequent recursive calls (e.g. `FinMem`(`Pi`(a', f), U) reducing to `FinMem`(a', U)) always have a structurally smaller first argument and a non- $\perp$ second argument, so the $\perp$ -swap never triggers again.

The auxiliary `FinMemFun`(g, a, f) recurses on the list g , calling `FinMem` on keys and values of g (strict subterms of the original first argument) with second arguments a and `Eval`(f, u_i) (derived from subterms of the original type code). After the initial $\perp$ -swap, all recursion is well-founded on the total size of the arguments.

6.8 Decidability

All the relations defined on finite elements are decidable.

LeCode. Decidability is built into the definition via the numeric function `leFinEl` : `Code` → `Code` → $\mathbb{N}$, which mirrors the structure of `LeCode` but returns a natural number (0 or positive). Soundness and completeness lemmas establish that `leFinEl`(u, v) > 0 if and only if `LeCode`(u, v).

Comp, Coherent, FinMem. The file `FinMemDecidable.agda` provides decision procedures for the remaining predicates, using `Dec` $A = A + (A \rightarrow \perp)$:

```
decComp : (u v : Code) → Dec(Comp(u, v))
decCoherent : (u : Code) → Dec(Coherent(u))
decFinMem : (u a : Code) → Dec(FinMem(u, a))
```

Each decision procedure follows the same case structure as the corresponding definition, composing decisions through generic combinators for pairs (`dec-Pair`), disjunctions (`dec-Or`), and implications (`dec-impl`, using decidability of the premise to either produce a witness or refute by applying the function to it). All proofs are postulate-free.

7 The logical relation

The logical relation connects the domain-theoretic model to the syntax. It is defined by mutual recursion on codes, with six mutually defined predicates. We describe the main ones.

7.1 Term validity

$\text{Val } \Gamma \ M \ A \ u \ a$ asserts that term M of type A in context Γ is valid at value code u and type code a . The definition is by case analysis on the pair (u, a) :

- $\text{Val } \Gamma \ M \ A \ u \ \perp = \top$ (trivially valid at bottom type).
- At the universe: $\text{Val } \Gamma \ M \ A \ u \ \mathbf{U}$ is non-trivial only when $u \in \{\mathbf{U}, \text{Prop}, \text{Pi}(b, f), \text{Sigma}(b, f)\}$, in which case it requires $\text{ValTy } \Gamma \ A \ \mathbf{U}$ (the type A is valid at $\mathbf{U}$) together with $\text{ValTy } \Gamma \ M \ u$ (the term M is a valid type code u).
- At a Pi-type: $\text{Val } \Gamma \ M \ A \ (\lambda g) \ (\text{Pi}(b, f))$ requires $\text{ValTy } \Gamma \ A \ (\text{Pi}(b, f))$ together with application data: for each $(u_0, v_0) \in g$ and argument N with $\Gamma \vdash N : A_0$ and $\text{Val } \Gamma \ N \ A_0 \ u_0 \ b$, one has $\text{Val } \Gamma \ (M \ N) \ (B_0[N]) \ v_0 \ \text{Eval}(f, u_0)$.
- At a Sigma-type: $\text{Val } \Gamma \ M \ A \ (u', v') \ (\text{Sigma}(b, f))$ requires $\text{ValTy } \Gamma \ A \ (\text{Sigma}(b, f))$ together with validity of both projections (see Section 11 below).
- All other combinations are trivially $\top$.

7.2 Type validity

$\text{ValTy } \Gamma \ M \ a$ asserts that M is a valid type at code a . The interesting cases are:

- $\text{ValTy } \Gamma \ M \ \mathbf{U}$ requires a head reduction $M \rightsquigarrow^* \mathbf{U}$ together with $\Gamma \vdash M = \mathbf{U} : \mathbf{U}$.
- $\text{ValTy } \Gamma \ M \ (\text{Pi}(b, f))$ unpacks to specific types A, B with $M \rightsquigarrow^* \Pi(x:A)B$, typing evidence $\Gamma \vdash A : \mathbf{U}$ and $\Gamma, x:A \vdash B : \mathbf{U}$, domain validity $\text{ValTy } \Gamma \ A \ b$, and *edge functions*: for each $(u, v) \in f$ and well-typed argument N , $\text{ValTy } \Gamma \ B[N] \ v$.
- $\text{ValTy } \Gamma \ M \ (\text{Sigma}(b, f))$ is analogous, with $M \rightsquigarrow^* \Sigma(x:A)B$.

7.3 Equality

$\text{EqVal } \Gamma \ M \ N \ A \ u \ a$ bundles Val for both M and N together with an equality witness. It follows the same case structure. $\text{EqValTy } \Gamma \ M \ N \ a$ is the corresponding type-level equality.

7.4 Edge functions and selections

The records for ValTy at $\text{Pi}(b, f)$ and $\text{Sigma}(b, f)$ contain *edge functions* that witness how the codomain type varies with the input. For instance, the value edge in the Pi case has the form:

for all (u, v) selected from f , and N with $\Gamma \vdash N : A$ and $\text{Val } \Gamma \ N \ A \ u \ b$, one has $\text{ValTy } \Gamma \ B[N] \ v$.

Here (u, v) is *selected* from f in the sense of $\text{Selection}(f, u, v)$: each entry (u_i, v_i) in f is either skipped (contributing $(\perp, \perp)$) or taken (combining via $\sqcup$ with a compatibility proof), yielding a pair (u, v) that satisfies $u \leq \text{Eval}(f, u)$ and is built from subterms of f .

Similarly, the application data in Val at $(\lambda g, \text{Pi}(b, f))$ requires: for all (u, v) selected from g and well-typed argument N with $\text{Val } \Gamma \ N \ A_0 \ u \ b$, one has $\text{Val } \Gamma \ (M \ N) \ B_0[N] \ v \ \text{Eval}(f, u)$.

7.5 Termination of the logical relation

The four mutually defined predicates Val , EqVal , ValTy , EqValTy and their associated record types form a single mutual block, marked **TERMINATING** in the formalisation. The well-foundedness argument is more subtle than for the domain-level predicates: the recursion cannot be measured by the type code a alone.

The difficulty at the universe. Consider $\text{Val } \Gamma M A (\text{Pi}(a', f')) U$. This reduces to $\text{ValTy } \Gamma A U$ paired with $\text{ValTy } \Gamma M (\text{Pi}(a', f'))$. The second component recurses at type code $\text{Pi}(a', f')$, which is not a subterm of the original type code U —it is part of the *value code*. Moreover a' can itself be U , so the recursion is not simply structural on either code.

The measure: depth of $\text{FinMem}(u, a)$. The relation $\text{Val } \Gamma M A u a$ is only meaningful when $\text{FinMem}(u, a)$ holds—this is a precondition maintained by the adequacy proof. The correct well-foundedness measure is the *size of the $\text{FinMem}(u, a)$ derivation*: at each recursive call, the corresponding membership proof is a strict sub-derivation.

We trace this through the cases:

- $a = \perp$: Val returns $\top$, no recursion.
- $a = U$, $u = \text{Pi}(a', f')$: The membership $\text{FinMem}(\text{Pi}(a', f'), U)$ consists of $\text{FinMem}(a', U)$, $\text{FinMemAllU}(f', a')$, and $\text{CoherentFunTail}(f')$. The definition produces $\text{ValTy } \Gamma M (\text{Pi}(a', f'))$, which unfolds to RValTyPi containing:
 - $\text{ValTy } \Gamma A_0 a'$, with membership $\text{FinMem}(a', U)$ —a sub-derivation of the original $\text{FinMem}(\text{Pi}(a', f'), U)$. But this call is at (a', U) which itself unfolds further: if $a' = \text{Pi}(a'', f'')$, the membership $\text{FinMem}(a'', U)$ is again strictly smaller. Since the membership derivation is finite, this chain terminates.
 - Edge functions producing $\text{ValTy } \Gamma B[N] v$ for (u_0, v_0) selected from f' , where the membership evidence for v_0 at U comes from $\text{FinMemAllU}(f', a')$ —again a sub-derivation.

The case $u = \text{Sigma}(a', f')$ at U is analogous.

- $a = \text{Pi}(b, f)$, $u = \lambda g$: The membership $\text{FinMem}(\lambda g, \text{Pi}(b, f))$ consists of $\text{FinMemFun}(g, b, f)$, $\text{CoherentFun}(g)$, and $\text{FinMem}(\text{Pi}(b, f), U)$. The application data requires $\text{Val } \Gamma (M N) B_0[N] v \text{ Eval}(f, u_0)$ for (u_0, v_0) selected from g . The membership $\text{FinMem}(v_0, \text{Eval}(f, u_0))$ is derivable from $\text{FinMemFun}(g, b, f)$ —a sub-derivation. The bundled $\text{ValTy } \Gamma A (\text{Pi}(b, f))$ recurses at the membership $\text{FinMem}(\text{Pi}(b, f), U)$, which is a component of the original membership.
- $a = \text{Sigma}(b, f)$, $u = (u', v')$: The membership $\text{FinMem}((u', v'), \text{Sigma}(b, f))$ contains $\text{FinMem}(u', b)$ and $\text{FinMem}(v', \text{Eval}(f, u'))$. The recursive calls $\text{Val } \Gamma \text{fst}(M) A_0 u' b$ and $\text{Val } \Gamma \text{snd}(M) B_0[\text{fst}(M)] v' \text{ Eval}(f, u')$ use exactly these sub-derivations as their membership evidence.
- **All other** (u, a) : Val returns $\top$, no recursion.

Summary. The recursion is well-founded on the size of the $\text{FinMem}(u, a)$ derivation. At each recursive call—whether descending into domain codes, edge functions, application data, or projections—the corresponding membership evidence is a strict sub-derivation. The universe case, where $a = U$ and the recursion shifts to the value code u , is controlled by the fact that $\text{FinMem}(u, U)$ decomposes u into strictly smaller membership sub-proofs (domain code, codomain entries), even though u is not a subterm of U .

Positivity. The records (RValPi, RValTyPi, etc.) are marked `NO_POSITIVITY_CHECK` in the formalisation because `Val` and `EqVal` appear in negative position within the edge functions (as hypotheses: “for all N with $\text{Val } \Gamma \ N \ A \ u \ b, \dots$ ”). This is sound because the edge functions are *stored data*, not computationally inspected by the definition of `Val` itself—they are produced by the adequacy proof, which constructs them by induction on typing derivations.

7.6 Adequacy

The *adequacy theorem* (Theorem 2 of Coquand–Huber 2018, extended to Σ) states that typability implies validity. In the two-substitution form:

Theorem 1 (Adequacy). *If $\Gamma \vdash M : A$, then for any coherent environment ρ with a valid substitution σ into a context H , and any codes u, a such that M evaluates to u at ρ and A evaluates to a at ρ with $\text{FinMem}(u, a)$, we have $\text{Val } H \ \sigma(M) \ \sigma(A) \ u \ a$.*

The proof proceeds by mutual induction on the typing derivation, with a parallel statement for conversion ($\Gamma \vdash M = N : A$ implies `EqVal`) and a two-substitution variant (M under two related substitutions gives `EqVal`).

The injectivity results and subject reduction below are *corollaries* of adequacy: one evaluates at the trivial bottom environment $\rho_\perp$ to extract head-reduction and conversion data from the logical relation.

8 Pi injectivity

Theorem 2 (Pi injectivity — Corollary 6). *If $\Gamma \vdash A_0 = \Pi(x:B_1)F_1 : \mathbb{U}$, then there exist B_0, F_0 such that*

1. $A_0 \rightsquigarrow^* \Pi(x:B_0)F_0$,
2. $\Gamma \vdash B_0 = B_1 : \mathbb{U}$,
3. $\Gamma, x:B_0 \vdash F_0 = F_1 : \mathbb{U}$.

Corollary 3 (Pi–Pi injectivity). *If $\Gamma \vdash \Pi(x:A_0)B_0 = \Pi(x:A_1)B_1 : \mathbb{U}$, then*

$$\Gamma \vdash A_0 = A_1 : \mathbb{U} \quad \text{and} \quad \Gamma, x:A_0 \vdash B_0 = B_1 : \mathbb{U}.$$

Proof outline. The proof proceeds by domain-theoretic adequacy:

1. From the conversion, extract $\Gamma \vdash A_0 : \mathbb{U}$ by presupposition.
2. Apply conversion soundness at the bottom environment $\rho_\perp$ to transfer $\text{EvalRel}(\Pi B_1 \ F_1, \rho_\perp, \text{Pi}(\perp, \text{nil}))$ to $\text{EvalRel}(A_0, \rho_\perp, \text{Pi}(\perp, \text{nil}))$.
3. Apply bundled equality adequacy (`adequacyEqSub2`) to obtain `EqVal2` at the $\text{Pi}(\perp, \text{nil})$ code, which unfolds to `EqValTyPi2` containing the head reduction and domain/codomain conversions.
4. Since Π is a head normal form, reduction uniqueness (`Red-unique-Pi`) gives exact equalities.

□

9 Sigma injectivity

Theorem 4 (Sigma injectivity). *If $\Gamma \vdash A_0 = \Sigma(x:B_1)F_1 : \mathbb{U}$, then there exist B_0, F_0 such that*

1. $A_0 \rightsquigarrow^* \Sigma(x:B_0)F_0$,
2. $\Gamma \vdash B_0 = B_1 : \mathbb{U}$,
3. $\Gamma, x:B_0 \vdash F_0 = F_1 : \mathbb{U}$.

Corollary 5 (Sigma-Sigma injectivity). *If $\Gamma \vdash \Sigma(x:A_0)B_0 = \Sigma(x:A_1)B_1 : \mathbb{U}$, then*

$$\Gamma \vdash A_0 = A_1 : \mathbb{U} \quad \text{and} \quad \Gamma, x:A_0 \vdash B_0 = B_1 : \mathbb{U}.$$

Proof outline. The proof follows the same strategy as Pi injectivity:

1. Extract $\Gamma \vdash A_0 : \mathbb{U}$ by presupposition.
2. Transfer $\text{EvalRel}(\Sigma B_1 F_1, \rho_\perp, \text{Sigma}(\perp, \text{nil}))$ to $\text{EvalRel}(A_0, \rho_\perp, \text{Sigma}(\perp, \text{nil}))$ via conversion soundness.
3. Apply bundled equality adequacy to obtain EqValTySigma2 containing head reduction and domain/codomain conversions.
4. Since Σ is a head normal form, reduction uniqueness (**Red-unique-Sigma**) gives exact equalities.

□

10 Subject reduction

As an application of Pi and Sigma injectivity, we prove subject reduction for single-step head reduction. The proof requires two mutually defined theorems due to the Snd case.

Theorem 6 (Subject reduction and subject conversion). *The following are proved by mutual induction on the typing derivation:*

1. **Subject reduction.** *If $\Gamma \vdash M : A$ and $M \rightsquigarrow_1 N$, then $\Gamma \vdash N : A$.*
2. **Subject conversion.** *If $\Gamma \vdash M : A$ and $M \rightsquigarrow_1 N$, then $\Gamma \vdash M = N : A$.*

The mutual definition is necessary because of the SND case: when $M \rightsquigarrow_1 M'$ and we have $\Gamma \vdash \text{snd}(M) : B[\text{fst}(M)]$, subject reduction for $\text{snd}(M) \rightsquigarrow_1 \text{snd}(M')$ must produce $\Gamma \vdash \text{snd}(M') : B[\text{fst}(M)]$. But the direct application of SND gives type $B[\text{fst}(M')]$. To bridge the gap, we need $\Gamma \vdash \text{fst}(M) = \text{fst}(M') : A$, which requires subject conversion (not just subject reduction) applied to M .

Proof outline. By mutual induction on the typing derivation, case-splitting on the head reduction step.

Subject reduction cases:

- CONV and PROP-U: peel off and recurse.
- APP with HR-APP ($f \rightsquigarrow_1 f'$): induction gives $\Gamma \vdash f' : \Pi(x:A)B$; rebuild with APP.

- APP with HR-BETA $((\lambda(x:A')M') a \rightsquigarrow_1 M'[a])$: the key case. Use Pi injectivity to extract the body typing $\Gamma, x:A_0 \vdash M' : B_0$ from the lambda derivation (walking through CONV layers), then apply substitution. The helper lemma **ty-Lam-body** performs this extraction.
- FST with HR-FST $(M \rightsquigarrow_1 M')$: induction gives $\Gamma \vdash M' : \Sigma(x:A)B$; rebuild with FST.
- FST with HR-BETA-FST $(\text{fst}(\text{pair}(M, N)) \rightsquigarrow_1 M)$: use Sigma injectivity to extract $\Gamma \vdash M : A$ from the MkPair derivation. The helper **ty-MkPair-components** performs this extraction.
- SND with HR-SND $(M \rightsquigarrow_1 M')$: induction on subject conversion gives $\Gamma \vdash M = M' : \Sigma(x:A)B$, hence $\Gamma \vdash \text{fst}(M) = \text{fst}(M') : A$ by FST-CONG. Subject reduction gives $\Gamma \vdash \text{snd}(M') : B[\text{fst}(M')]$; type conversion via $\text{fst}(M) = \text{fst}(M')$ yields $\Gamma \vdash \text{snd}(M') : B[\text{fst}(M)]$.
- SND with HR-BETA-SND $(\text{snd}(\text{pair}(M, N)) \rightsquigarrow_1 N)$: use Sigma injectivity and **ty-MkPair-components** to extract $\Gamma \vdash N : B[M]$, then convert using $\Gamma \vdash M = \text{fst}(\text{pair}(M, N)) : A$ (the symmetric Beta-Fst).

Subject conversion follows from subject reduction: given $\Gamma \vdash M : A$ and $M \rightsquigarrow_1 N$, subject reduction gives $\Gamma \vdash N : A$, and the corresponding conversion rule for the head reduction step (β , β -fst, β -snd, or congruence) gives $\Gamma \vdash M = N : A$.

Key helper lemmas:

- **ty-Lam-body**: If $\Gamma \vdash \lambda(x:A')M : \Pi(x:A)B$, then $\Gamma, x:A \vdash M : B$. Proved by walking through CONV (using Pi injectivity) and eliminating PROP-U (since U cannot head-reduce to Π).
- **ty-MkPair-components**: If $\Gamma \vdash \text{pair}(M, N) : \Sigma(x:A)B$, then $\Gamma \vdash M : A$ and $\Gamma \vdash N : B[M]$. Proved by walking through CONV (using Sigma injectivity) and eliminating PROP-U.

□

11 Validity for Sigma types

We describe the extension of the logical relation (validity) to handle Σ -types. The key new difficulty compared to Π -types is in the head-reduction transport lemma.

11.1 Codes and projections

The set **Code** of codes includes:

$$u, v ::= \perp \mid \mathbf{U} \mid \mathbf{Prop} \mid \lambda g \mid \mathbf{Pi}(b, f) \mid \mathbf{Sigma}(b, f) \mid (u, v)$$

A code $\mathbf{Sigma}(b, f)$ represents a Σ -type whose domain has code b and whose codomain is the finite function f . A code (u, v) represents a pair value.

We use a uniform projection notation for both terms and codes:

$$M.1, \quad M.2 \quad (\text{first and second projection of a term})$$

$$w.1, \quad w.2 \quad (\text{first and second projection of a code})$$

On codes, the projections are defined by:

$$\begin{aligned} (u, v).1 &= u, & w.1 &= \perp \text{ otherwise} \\ (u, v).2 &= v, & w.2 &= \perp \text{ otherwise} \end{aligned}$$

11.2 Term validity at Σ

Definition 7 (Term validity at Σ). We define $\text{Val } \Gamma M T w \text{ Sigma}(b, f)$ to hold when:

1. $\text{ValTy } \Gamma T (\text{Sigma}(b, f))$ (type validity of T), which includes $T \rightsquigarrow^* \Sigma(x:A)B$ with $\Gamma \vdash A : \mathbb{U}$ and $\Gamma, x:A \vdash B : \mathbb{U}$, together with the edge functions;
2. $\text{Val } \Gamma M.1 A w.1 b$ (validity of the first projection);
3. $\text{Val } \Gamma M.2 B[M.1] w.2 f(w.1)$ (validity of the second projection).

When w is not a pair (u, v) , we have $w.1 = \perp$ and $w.2 = \perp$. Since Val at code $\perp$ is trivially $\top$, conditions (2) and (3) are automatically satisfied. This means there is no need for separate case analysis on whether w is a pair or not: the definition is *uniform* in w .

Compared to the Π -case, where term validity at $\text{Pi}(b, f)$ is defined for $w = \lambda g$ and stores $\text{Val } \Gamma (M N) (B[N]) v (f(u))$, the type $B[N]$ does not mention M . For Σ , the second component's type $B[M.1]$ *does* mention M . This is the source of the difficulty in head-reduction transport.

11.3 Type validity at Σ

Definition 8 (Type validity at Σ). $\text{ValTy } \Gamma T (\text{Sigma}(b, f))$ holds when:

- $T \rightsquigarrow^* \Sigma(x:A)B$ and $\Gamma \vdash T = \Sigma(x:A)B : \mathbb{U}$;
- $\Gamma \vdash A : \mathbb{U}$ and $\Gamma, x:A \vdash B : \mathbb{U}$;
- $\text{ValTy } \Gamma A b$ (domain type validity);
- For each $(u, v) \in f$ and N with $\Gamma \vdash N : A$ and $\text{Val } \Gamma N A u b$, we have $\text{ValTy } \Gamma B[N] v$ (edge function for values);
- Similarly for the equality edge.

12 Type-equality transport

Lemma 9 (Transport along type equality). For all codes u and a :

$$\text{EqValTy } \Gamma C C' a \implies \text{Val } \Gamma N C u a \implies \text{Val } \Gamma N C' u a$$

Proof. By induction on a . The type parameter C appears in $\text{Val } \Gamma N C u a$ only in two cases: $a = \text{Pi}(b, f)$ with $u = \lambda g$, and $a = \text{Sigma}(b, f)$ with $u = (u', v')$. In these cases the validity record stores a head reduction $C \rightsquigarrow^* \Pi(x:A)B$ or $C \rightsquigarrow^* \Sigma(x:A)B$, and $\text{EqValTy } \Gamma C C' a$ provides the corresponding data for C' , allowing the record to be rebuilt. In all other cases—including $a = \mathbb{U}$, where Val depends on N but not on C —the result is immediate. $\square$

This lemma plays a central role in the Σ case of head-reduction transport (Lemma 10, Step 4), where it transports Val from type $B[M'.1]$ to type $B[M.1]$. For Π -types alone, this lemma is not needed in head-reduction transport.

13 Head-reduction transport

The central lemma for adequacy is:

Lemma 10 (Head-reduction produces equality). *Given:*

- $M \rightsquigarrow^* M'$ and $\Gamma \vdash M = M' : T$,
- $\text{Val } \Gamma \ M' \ T \ u \ a$ (validity of the reduct),

we can conclude:

$$\text{EqVal } \Gamma \ M' \ M \ T \ u \ a$$

which bundles $\text{Val } \Gamma \ M \ T \ u \ a$ and $\text{Val } \Gamma \ M' \ T \ u \ a$ and the equality $\text{EqVal } \Gamma \ M' \ M \ T \ u \ a$.

The return type is **EqVal** (not just **Val**). This is essential for the Σ case: the equality at the first projection is needed to transport the second projection's type.

13.1 The Π case

For $a = \text{Pi}(b, f)$ and $u = \lambda g$, the validity data for M' stores, for each $(u_0, v_0) \in g$ and argument N :

$$\text{Val } \Gamma \ (M' \ N) \ (B_0[N]) \ v_0 \ (f(u_0))$$

Since $M \rightsquigarrow^* M'$, we have $M \ N \rightsquigarrow^* M' \ N$. The type $B_0[N]$ does not mention M , so the induction hypothesis applies directly at code $(v_0, f(u_0))$:

$$\text{EqVal } \Gamma \ (M' \ N) \ (M \ N) \ (B_0[N]) \ v_0 \ (f(u_0))$$

No type transport is needed.

13.2 The Σ case

For $a = \text{Sigma}(b, f)$, we work with the uniform definition. The validity data for M' includes:

$$\text{Val } \Gamma \ M'.1 \ A \ w.1 \ b \tag{1}$$

$$\text{Val } \Gamma \ M'.2 \ B[M'.1] \ w.2 \ f(w.1) \tag{2}$$

We need to produce the same data for M , plus equality.

Step 1: First projection. From $M \rightsquigarrow^* M'$ we get $M.1 \rightsquigarrow^* M'.1$. From $\Gamma \vdash M = M' : \Sigma(x:A)B$ we derive (by the congruence rule for projections):

$$\Gamma \vdash M.1 = M'.1 : A$$

Together with (1), the induction hypothesis at code $(w.1, b)$ gives:

$$\text{EqVal } \Gamma \ M'.1 \ M.1 \ A \ w.1 \ b \tag{3}$$

This bundles $\text{Val } \Gamma \ M.1 \ A \ w.1 \ b$.

Step 2: Type transport via the edge function. The equality (3) and the equality edge function from **ValTy** at $\text{Sigma}(b, f)$ give:

$$\text{EqValTy } \Gamma \ B[M'.1] \ B[M.1] \ f(w.1) \tag{4}$$

This is the key step: the *equality* edge function converts the equality of *values* ($M'.1 = M.1$) into an equality of *types* ($B[M'.1] = B[M.1]$). This step has no analogue in the Π case.

Note that the *value* edge (which only takes **Val** data) would not suffice here: we genuinely need the *equality* at the first projection to produce the type equality for the transport. This is why Lemma 10 must return **EqVal** rather than just **Val**.

Step 3: Second projection. From $M \rightsquigarrow^* M'$ we get $M.2 \rightsquigarrow^* M'.2$. The congruence rule for the second projection, applied to $\Gamma \vdash M' = M : \Sigma(x:A)B$ (note the swap, so that $M'.1$ appears in the type), gives:

$$\Gamma \vdash M'.2 = M.2 : B[M'.1]$$

Together with (2), the induction hypothesis at code $(w.2, f(w.1))$ gives:

$$\text{EqVal } \Gamma \ M'.2 \ M.2 \ B[M'.1] \ w.2 \ f(w.1) \quad (5)$$

In particular, $\text{Val } \Gamma \ M.2 \ B[M'.1] \ w.2 \ f(w.1)$.

Step 4: Transport. The validity data for M at Σ requires:

$$\text{Val } \Gamma \ M.2 \ B[M.1] \ w.2 \ f(w.1)$$

We have **Val** at $B[M'.1]$ from Step 3, and **EqValTy** between $B[M'.1]$ and $B[M.1]$ from Step 2. Lemma 9 gives $\text{Val } \Gamma \ M.2 \ B[M.1] \ w.2 \ f(w.1)$.

14 Bundling requirements

For Π , $\mathbf{U}$, and **Prop** alone, the following simpler choices suffice:

- Head reduction (**Red**) stores only $T \rightsquigarrow^* \Pi(x:A)B$, without the convertibility judgment $\Gamma \vdash T = \Pi(x:A)B : \mathbf{U}$.
- The head-reduction transport lemma can return just **Val** (validity of the expanded term), without the equality **EqVal**.
- Term validity $\text{Val } \Gamma \ M \ T \ u \ a$ need not be bundled with type validity $\text{ValTy } \Gamma \ T \ a$.

For Σ -types, all three must be strengthened:

1. **Red must bundle convertibility.** The head-reduction transport for Σ needs to derive $\Gamma \vdash M.1 = M'.1 : A$ (via the congruence rule for projections). This congruence rule requires $\Gamma \vdash M = M' : \Sigma(x:A)B$, which is obtained from the input $\Gamma \vdash M = M' : T$ by converting the type using $\Gamma \vdash T = \Sigma(x:A)B : \mathbf{U}$ —the convertibility stored in the type validity record alongside the head reduction $T \rightsquigarrow^* \Sigma(x:A)B$.
2. **Head-reduction transport must return EqVal.** The equality **EqVal** at the first projection is needed to invoke the equality edge function (Step 2), which produces the type equality $\text{EqValTy } \Gamma \ B[M'.1] \ B[M.1] \ f(w.1)$ used in the transport (Step 4).
3. **Val must be bundled with ValTy.** Step 2 uses the equality edge function from $\text{ValTy } \Gamma \ T \ (\text{Sigma}(b, f))$. This must be accessible from within the term validity data. Therefore $\text{Val } \Gamma \ M \ T \ w \ (\text{Sigma}(b, f))$ must include $\text{ValTy } \Gamma \ T \ (\text{Sigma}(b, f))$ as a component.

14.1 Comparison with Π

	Π	Σ
Value code	λg	any w (uniform via $w.1/w.2$)
Stored data	$\text{Val } (M\ N)\ B[N]\ \dots$	$\text{Val } M.1\ A\ \dots$ and $\text{Val } M.2\ B[M.1]\ \dots$
Type depends on M ?	No ($B[N]$)	Yes ($B[M.1]$)
HeadRed transport	Direct IH	IH + edge + transport
Returns EqVal?	Useful	Essential
Red bundles ConvTm?	Not needed	Needed (for conv-Fst/conv-Snd)
Val bundles ValTy?	Not needed	Needed (for equality edge)

15 Agda formalization

The entire development is formalised in Agda with flags `--without-K` `--exact-split` (no `--type-in-type`) and `0` `postulates` throughout.

Core files:

- `RawSyntax.agda` — raw terms (Π , Σ , λ , App, MkPair, Fst, Snd, U, Prop), substitution, weakening.
- `TypingRules.agda` — typing and conversion judgments.
- `PaperSemantics.agda` — trusted kernel: finite elements (`FinEl`, `FinFun`), information ordering, coherence, finite membership. No postulates.
- `RawSemantics.agda` — selection-based evaluation relation. No postulates.
- `TypingSemantics.agda` — typing and conversion soundness. 0 postulates.
- `Reduction.agda` — head reduction (`HeadRed1`, `Red`).

Validity stack (Validity5 series):

- `Validity5Core.agda` — base definitions of `Val2`/`EqVal2` with record types.
- `Validity5DownUpRestrict.agda` — monotonicity: downward, upward, and restriction in the code argument.
- `Validity5Fwd.agda` — forward lemmas.
- `Validity5SymTrans.agda` — symmetry and transitivity for `EqVal2`.
- `Validity5Sup.agda` — sup closure.
- `Validity5Lemmas.agda` — collected utility lemmas.

Main results:

- `Adequacy5.agda` + `Adequacy5Cases.agda` — adequacy proof (two-substitution form). 0 postulates.
- `Adequacy5Helpers.agda`, `Adequacy5HeadRed.agda` — helper lemmas for adequacy.
- `Injectivity5.agda` — Π and Σ injectivity. 0 postulates.
- `SubjectReduction5.agda` — subject reduction + subject conversion (mutual). 0 postulates.

References

- [1] T. Coquand and S. Huber, “An adequacy theorem for dependent type theory,” *Archive for Mathematical Logic*, vol. 57, pp. 649–676, 2018.